

Reinforcement Learning SoSe 2025 - Task 2

Bericht

Tsvetan Stanchev

1) Einleitung

Die zweite Aufgabe bestand darin, die Umgebung CarRacing-v3 aus der OpenAI Gym-Bibliothek mit einem eigenen Deep-Q-Learning-Agenten erfolgreich zu lösen. Ziel war es, durch gezieltes Training eine Strategie (Policy) zu lernen, mit der das Fahrzeug konstant eine kumulierte Belohnung von über 500 Punkten pro Episode erreichen kann. Eine zusätzliche Herausforderung bestand darin, dies in möglichst wenigen Episoden zu schaffen.

Zur Lösung der Aufgabe wurde ein modularer Ansatz in Python implementiert, welcher Deep Q-Learning mit einem Convolutional Neural Network (CNN) nutzt und die Hyperparameter über die Optimierungsbibliothek Optuna automatisch abstimmt.

2) Implementierung

A) Systemaufbau

Das Projekt wurde in einzelne Module unterteilt, um Lesbarkeit, Wiederverwendbarkeit und Wartbarkeit zu verbessern. Die wichtigsten Dateien sind:

- main.py – Einstiegspunkt für Training und Evaluation
- train.py – Trainingspipeline und Optuna-Zieldefinition
- evaluate.py – Evaluierung des besten Modells
- model.py – Definition des Deep Q-Networks
- replay_buffer.py – Replay Buffer zur Stabilisierung des Lernprozesses
- utils.py – Hilfsfunktionen wie train_step() und preprocess()
- config.py – Zentrale Konfigurationsparameter (z. B. Aktionsraum, Episodenzahl)

B) Modellarchitektur

Als Architektur wurde ein Convolutional Neural Network (CNN) verwendet, das Bildbeobachtungen mit 96×96 Pixeln verarbeitet. Die Netzwerkstruktur orientiert sich am klassischen DQN-Ansatz mit drei Faltungsschichten und zwei voll verbundenen Schichten:

- Conv2D → ReLU → Conv2D → ReLU → Conv2D → ReLU → Flatten
- Dense(4096 → 512) → ReLU → Dense(512 → action_dim)

Das Netzwerk gibt Q-Werte für einen diskreten Aktionsraum mit 5 definierten Steuerungsbefehlen aus (Lenken, Gas, Bremse).

C) DQN

Als Grundlage wurde der klassische DQN-Ansatz verwendet, bei dem ein Convolutional Neural Network (CNN) auf Bildbeobachtungen trainiert wird und direkt Q-Werte für einen diskreten Aktionsraum ausgibt. Implementiert wurden alle zentralen Komponenten des Deep Q-Learning-Algorithmus: die Haupt-Q-Netzwerkstruktur, ein Zielnetzwerk (target network) zur Stabilisierung, sowie eine Replay-Buffer-Struktur zur zufälligen Stichprobenentnahme aus früheren Erfahrungen. Die Aktualisierung der Q-Werte erfolgt auf Basis der Bellman-Gleichung mit einem Mean-Squared-Error-Loss zwischen den aktuellen Q-Werten und dem erwarteten Zielwert. Das Target-Netzwerk wird in regelmäßigen Abständen synchronisiert.

D) Replay Buffer & Preprocessing

Ein ReplayBuffer mit einer Kapazität von 100.000 Einträgen wird verwendet, um Erfahrungen zu speichern und aus zufälligen Mini-Batches zu lernen. Dies hilft, Korrelationen in den Trainingsdaten zu reduzieren.

Die Beobachtungen (RGB-Bilder) werden mit preprocess() in das für PyTorch erwartete Format ($C \times H \times W$) gebracht.

E) 3.4 Training & Hyperparameter-Optimierung

Das Training erfolgt über eine klassische DQN-Lernschleife, wobei folgende Parameter durch Optuna optimiert werden:

- Lernrate (lr): Steuert die Schrittweite, mit der das neuronale Netz seine Gewichtungen bei jedem Update anpasst. Eine zu hohe Lernrate kann zu Instabilität führen, eine zu niedrige zu langsamer Konvergenz.
- Endwert für ϵ -Greedy (epsilon_final): Gibt an, wie explorativ der Agent in der späten Trainingsphase noch agiert. Niedrige Werte sorgen für exploitation, höhere für weiterhin etwas exploration.
- Zerfallsrate von ϵ (epsilon_decay): Bestimmt, wie schnell der ϵ -Wert während des Trainings von 1.0 auf den Endwert absinkt. Ein schneller Zerfall führt dazu, dass der Agent früh gezielt handelt, ein langsamer Zerfall fördert längere Erkundung.
- Diskontierungsfaktor (gamma): Gibt an, wie stark zukünftige Belohnungen im Vergleich zu aktuellen gewichtet werden. Werte nahe 1.0 bevorzugen langfristige Strategien.
- Mini-Batch-Größe (batch_size): Anzahl der zufällig ausgewählten Übergänge aus dem Replay Buffer, die in einem Trainingsschritt verwendet werden. Größere Batches führen zu stabileren Gradienten, benötigen aber mehr Speicher.
- Update-Frequenz des Target-Netzwerks (target_update): Gibt an, nach wie vielen Schritten das Zielnetzwerk mit den aktuellen Parametern des Hauptnetzwerks

synchronisiert wird. Eine gute Wahl verhindert zu starke Korrelationen und stabilisiert das Lernen.

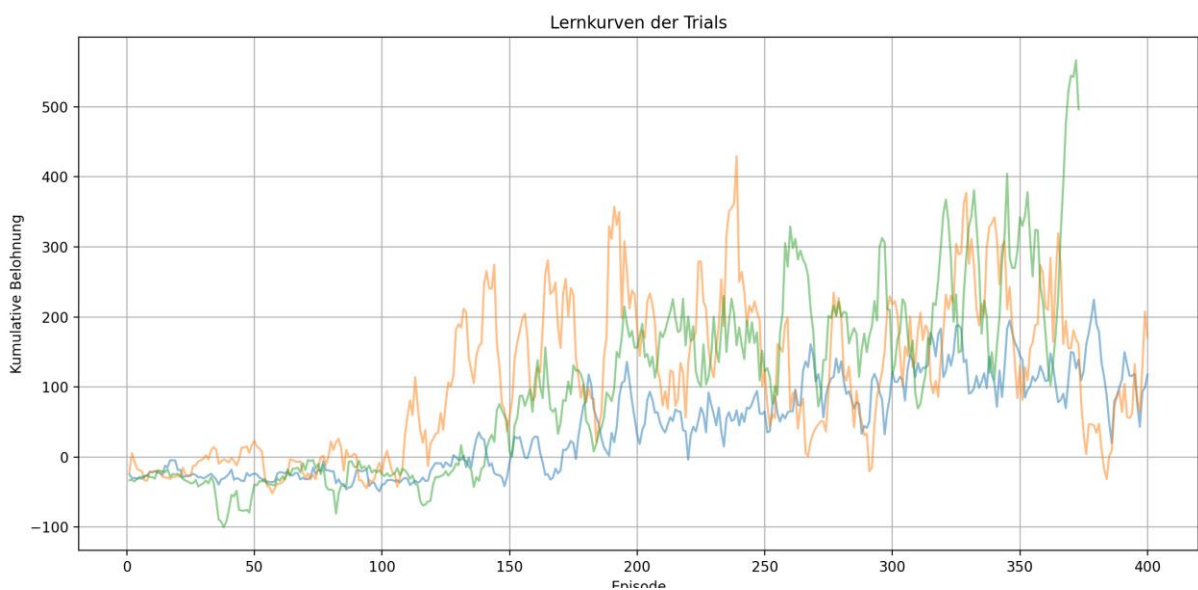
Optuna verwendet einen Zeit-Timeout, um das Training automatisch zu beenden (z. B. nach 8 Stunden). Zusätzlich wird ein Frühstopp ausgelöst, wenn die durchschnittliche Episodendauer über einen Schwellenwert hinausgeht oder eine Belohnung von über 500 erreicht wurde.

3) Experimente

Zur Durchführung der Experimente wurde die Umgebung mehrfach mit unterschiedlichen Parametern gestartet. Die Belohnungen pro Episode wurden automatisch protokolliert und am Ende eines Trials in einer Excel-Datei (optuna_results.xlsx) gespeichert. Für schnelle Testläufe wurden Parameter wie MAX_EPISODES = 3 und MAX_STEPS = 200 verwendet.

Die besten Ergebnisse wurden gespeichert in best_model.pth, welches für spätere Evaluierungen verwendet werden kann.

4) Ergebnisse



Die nebenstehende Grafik zeigt die geglätteten Lernkurven dreier Optuna-Trials über jeweils bis zu 400 Episoden. Während alle Agenten anfänglich eine niedrige Performance aufweisen, verbessert sich das Verhalten im Laufe des Trainings signifikant. Besonders hervorzuheben ist Trial 3, welcher bereits frühzeitig abgebrochen wurde, da der Agent in fünf aufeinanderfolgenden Episoden eine kumulierte Belohnung von über 500 Punkten erreichte. Dies zeigt, dass der optimierte Satz an Hyperparametern in diesem Fall zu einem besonders effizienten Lernverlauf geführt hat.

Die anderen beiden Trials benötigen deutlich mehr Episoden, um vergleichbare Ergebnisse zu erzielen – oder erreichen die Zielmarke gar nicht. Dieses Ergebnis unterstreicht die Notwendigkeit automatischer Hyperparameter-Optimierung mit Optuna, da kleine Änderungen im Parameterraum zu deutlich unterschiedlichen Lerneffekten führen können. Insgesamt belegt das Experiment, dass Deep Q-Learning in Verbindung mit CNNs, Experience Replay und Optuna ein wirksames Mittel ist, um komplexe kontinuierliche Steuerungsaufgaben wie CarRacing-v3 zu bewältigen.